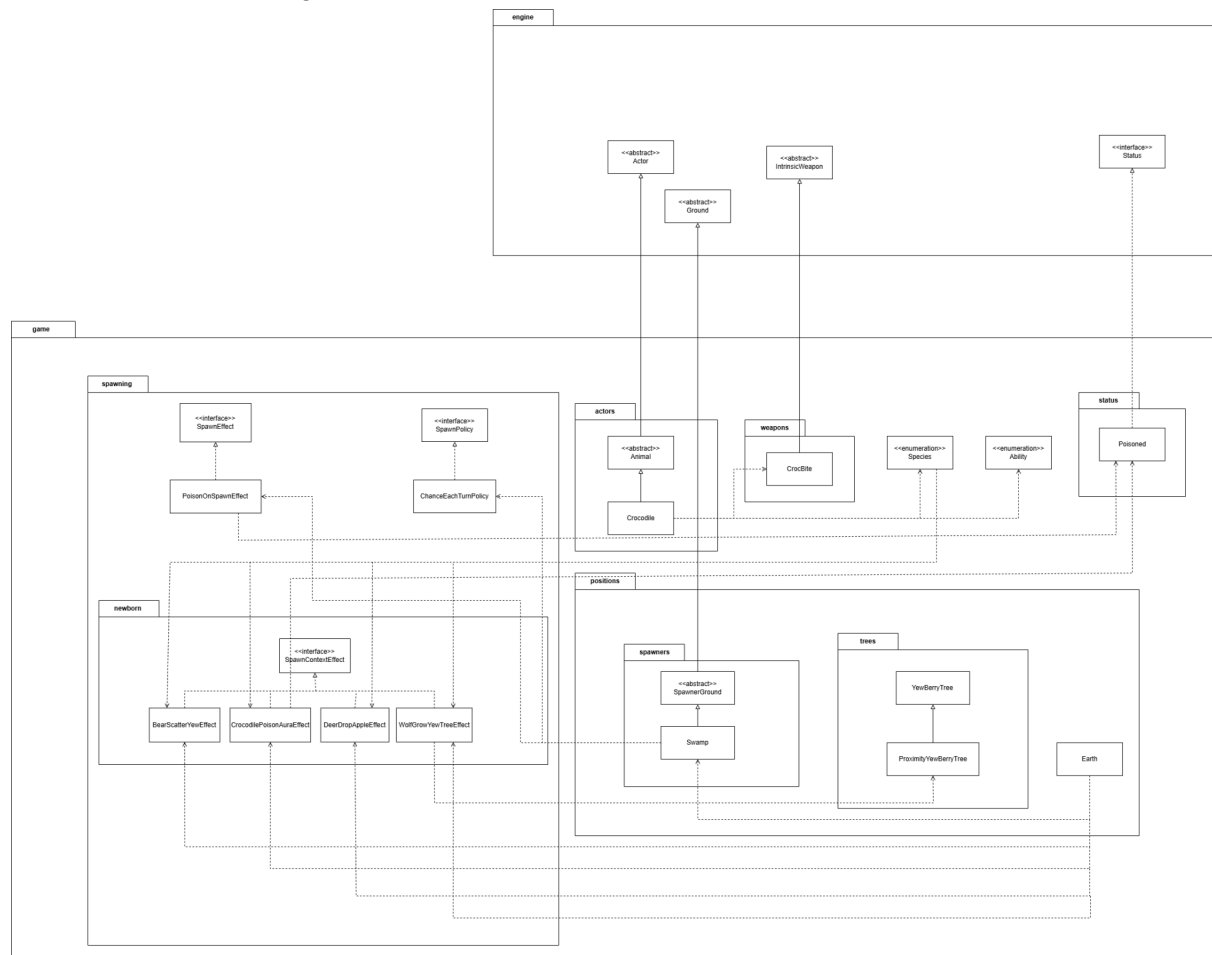


Design Diagram

REQ 2 UML Class Diagram:



Design Rational

Design 1:

All spawning, timing, probabilities, and after-spawn side effects are hard-coded directly inside each terrain class (Tundra, Meadow, Cave, Swamp). Each class:

- Keeps its own turn counter and RNG.
- Decides which species to spawn with big if/else or switch blocks.
- Applies all “extras” inline (e.g., deer to apple, bear to yewberries, wolf to grow tree, croc to AOE poison).

- Animals detect edible items with instanceof checks, and each edible item removes itself inside consumedBy(...).
- Seeking food uses a simple greedy Manhattan “step closer” rule.

No SpawnPolicy, no SpawnEffect, no SpawnContextEffect, no capabilities - just procedural glue in the terrain classes.

Pros	Cons
All logic is in one place per terrain - easy to “see everything”.	Becomes giant God classes; unreadable as features grow. Any new spawner or rule = copy/paste/modify multiple files (violates SRP/DRY/OCP).
No policies/effects/capabilities to understand; just if/else.	Tight coupling to specific species and items; every new animal/consumable requires editing many methods and recompiling wide surface area.
You can get “something that spawns” running quickly.	Maintenance nightmare: bug fixes require hunting duplicated code across spawners; high risk of inconsistent behaviour.
Terrain can see exits and drop items right there.	Side effects scattered: deer/bear/wolf/croc extras duplicated in four spawners; easy to forget one and break the spec in a map.
instanceof feels straightforward.	Closed to extension: adding a new edible needs more instanceof everywhere (action explosion, brittle code).
Item “cleans itself up”.	Asymmetry & bugs: inventory vs ground removal differs; easy to double-remove or forget to remove → re-eating items / ghost items.
Tiny code.	Gets stuck behind obstacles / oscillates; animals miss reachable food that’s not strictly closer each step.
You can test “one big method”.	Hard to unit test in isolation; no seams (policies/effects) to mock; lots of stateful condition branches.

Design 2:

Composable spawners: One generic SpawnerGround orchestrates spawning via plug-in parts: a SpawnPolicy (when), a list of species factories (what), and two effect types—SpawnEffect (mutate newborn) and SpawnContextEffect (location-aware side effects after placement).

Pluggable policies: Tundra uses chance-per-tick, Cave uses every-N-turns, Meadow uses gated chance-per-N; Swamp adds a nearby-actor gate with 50% chance and poisons its newborns (10×5).

Reusable extras (cross-spawner): On spawn: Deer drop 1 Apple on a random exit; Bear = 50% YewBerry on each exit; Wolf grow one special proximity-drop YewBerry tree; Crocodile poison nearby actors (3×10). All with clear display prints.

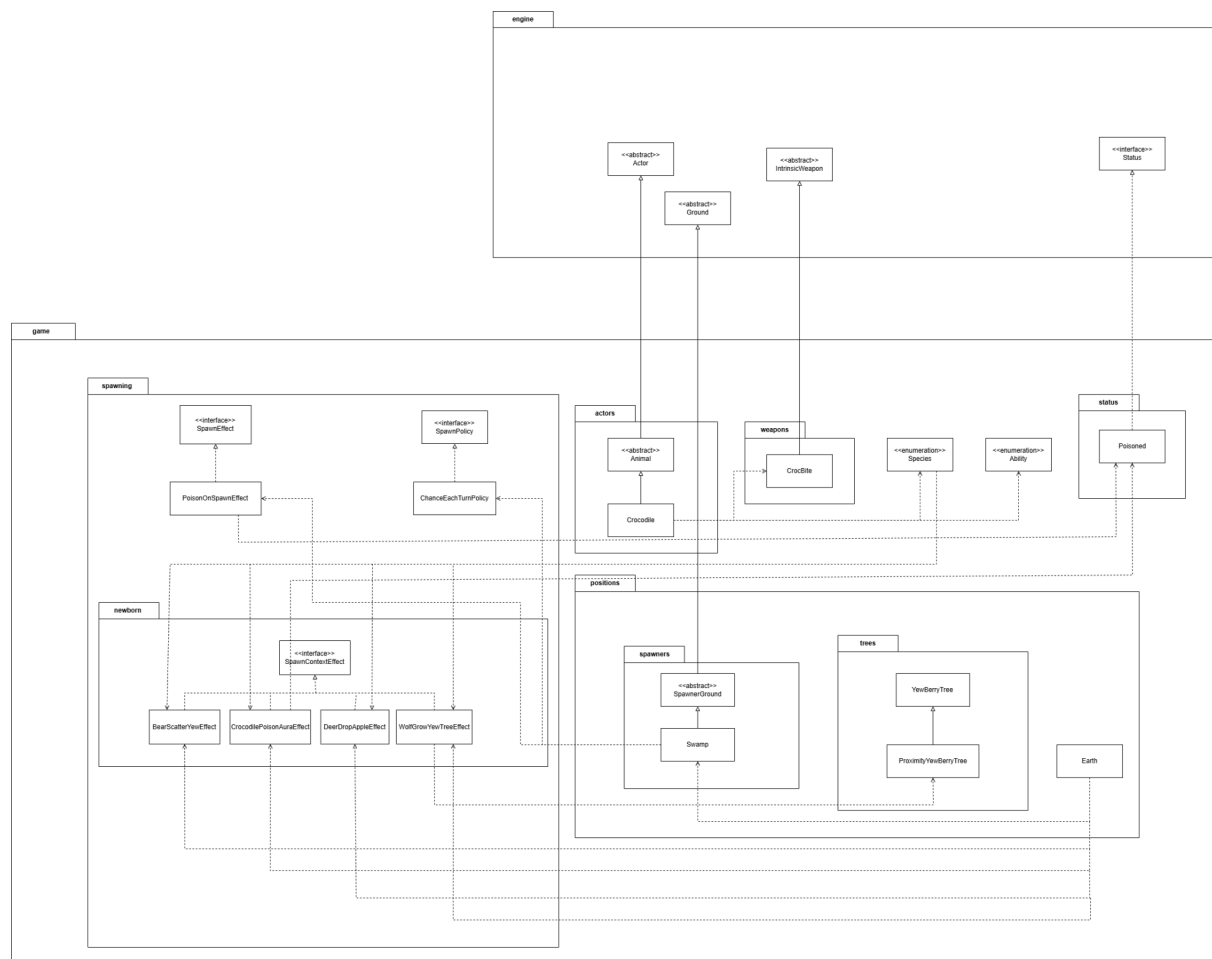
Clean consumption model: ConsumeAction applies Consumable effects (no instanceof) and then removes the item from ground or inventory—same flow for players and animals.

Animal behaviours: Animal exposes BehaviourHost and stores behaviours by priority; Meadow adds ForageBehaviour (eat here) and SeekConsumablesBehaviour (BFS toward food) at spawn time, without touching animal constructors.

OO principles: SRP/OCP/DIP throughout—spawner/policy/effects/behaviours are decoupled; new rules or species are added by composition, not edits.

Robust & demo-friendly: Engine add-actor exceptions are caught; effects run only after successful placement; terminal messages confirm every special rule firing.

Pros	Cons
Swap/add rules via Policies/Effects; re-use across terrains.	Slightly more types; we kept Swamp's gate inline to reduce moving parts.
New animals/spawn extras are <i>configuration</i> , not rewrites.	Need to remember to attach the context effects to new spawners — we centralised that in Earth.
Engine exceptions caught on add; effects run only after successful placement.	If placement fails (occupied), side effects don't run — by spec that's correct and avoids desync.
Forage on-tile first; BFS seek (radius-bound) reaches reachable food.	Radius needs tuning per map; we exposed it as a parameter and set sensible defaults.
Display prints for every special rule; easy to demo and test.	More logs in terminal — we kept messages concise.



The diagram above is my final design, which treats “spawning animals and triggering extra effects” as a set of small, composable building blocks. Instead of burying rules inside giant classes or long if/else chains, we separate the concerns and wire them together declaratively. At the centre is the abstract SpawnerGround, a ground tile that orchestrates the act of spawning on its location. Each concrete spawner—Tundra, Cave, Meadow, and the new Swamp—is a thin configuration: it chooses a SpawnPolicy (when to try spawning), registers one or more factories for species (e.g., `() -> new Bear(...)`), and attaches effects that either modify the newborn or act around the spawner’s tile. This composition-first approach is what lets us satisfy A3’s new behaviours (crocodiles, swamps, and the four “extras”) without touching engine code or rewriting animal classes.

SpawnerGround has one job: coordinate a spawn for its tile. On each tick, it delegates “should we spawn now?” to its SpawnPolicy. Policies are tiny strategies: “5% each turn” for tundra, “every N turns” for caves, “every 7 turns at 50%” for meadows, and “if any actor is nearby then 50%” for swamps. If the answer is yes, the spawner picks a species uniformly from its list of factories and creates the newborn. It then applies SpawnEffects (which mutate the newborn, such as the Meadow’s foraging behaviours or Swamp’s “poison newborn 10×5”) and places the actor on the tile. After a successful placement, it runs SpawnContextEffects that need the spawner location—for example, “drop an Apple on one exit” (deer), “each exit has 50% chance to drop a YewBerry” (bear), “grow one ProximityYewBerryTree” (wolf), and “poison nearby actors for 3 turns × 10 damage”

(crocodile). Each effect prints a clear, single-line message to the terminal so demonstrations are unambiguous.

This structure is intentionally aligned with core OOP principles:

Single Responsibility Principle. Every class does one thing. SpawnerGround only coordinates a spawn; it does not decide timing, encode species lists, or implement side effects. Policies only decide when. Effects only change the newborn or act around the tile. Behaviours are equally focused: one behaviour eats a consumable on the current tile; another walks one step toward the nearest reachable food. Because responsibilities are crisp, each piece is easier to read, test, and replace.

Open–Closed Principle. We extend behaviour by plugging in new policies, effects, or factories; we rarely modify existing classes. Adding crocodiles required registering a crocodile factory with the Forest’s Meadow and the Plains’ Tundra, then attaching the already-written “poison nearby actors” context effect. Adding the Swamp was a matter of composing a “nearby actor + 50%” policy with a “poison newborn 10×5” effect. The four cross-spawner extras—deer/apple, bear/yewberry scatter, wolf/proximity tree, croc/poison aura—are independent context effects that automatically apply across *all* spawners once attached.

Liskov Substitution Principle. All spawners are still Ground and all animals are still Actor; any place the engine expects these abstractions, our subclasses slot in without special treatment. Edible items remain ordinary Items that just expose a Consumable capability. No bespoke conditionals are needed to “treat them differently”.

Interface Segregation Principle. Our interfaces are small and purposeful. Consumable marks items with a consumedBy action, BehaviourHost lets us attach behaviours to animals at spawn time, and effects/policies are tiny contracts. Nothing is forced to implement more than it needs, avoiding “god interfaces”.

Dependency Inversion Principle. High-level orchestration depends on abstractions, not concretes. SpawnerGround depends on SpawnPolicy, SpawnEffect, SpawnContextEffect, and Supplier<Actor> rather than particular animals or rules. ConsumeAction depends on the Consumable capability instead of concrete item classes like Apple or YewBerry. This keeps coupling low and makes unit tests trivial to author.

The design is also deliberately DRY and consistent. All consumption—by player or animal—flows through a single ConsumeAction that applies the item’s effect and then removes the item from either inventory or ground. Spawning logic is unified inside SpawnerGround; differences between terrains live entirely in their policies and effect wiring. That means no duplicated “remove after eating” logic, and no per-terrain if/else cascades.

From a requirements perspective, this architecture makes each A3 change straightforward. The new **crocodile** is implemented as a normal Actor with bite and wander; wiring it into Forest’s Meadow and Plains’ Tundra is one-line factory registration per spawner. The **Swamp** is a slim subclass that composes a “nearby actor + 50%” policy with the “poison

newborn 10×5” SpawnEffect. The **extras** are SpawnContextEffects that run after placement so they can safely drop apples/yewberries, grow a proximity tree, and apply crocodile poison to neighbours—regardless of which spawner produced the animal. Terminal prints confirm each effect (“Deer spawned → dropped Apple at (x,y)”, “Bear spawned → YewBerry at exits...”, “Wolf spawned → Proximity YewBerry Tree at (x,y)”, “Crocodile spawned → poisoned N actors”), which is ideal for demos and marking.

Finally, the design is future-proof. To add “Canyon spawns Eagles every 10 turns; Eagles are wind-resistant”, we would create a Canyon spawner with EveryNTurnsPolicy(10), register an eagle factory, and attach a WindResistEffect. No engine edits, no rewiring of animals, and no regressions to the existing features. In short, this solution turns a complex, evolving rule set into small, reusable Lego blocks—clean, testable, and ready for whatever the next requirement brings.

What changed from A2 → A3, and why

1) Added a new animal: Crocodile

- Change: New Crocodile class (300 HP, bite 80 dmg @ 75% hit, Wander, WARMTH=55).
- Why: Directly required by A3. Implemented as a normal Actor subclass so it fits anywhere the engine expects an actor (LSP). No engine edits needed.

2) Extended existing spawners to include crocodiles

- Change: Forest Meadow now includes a Crocodile factory; Plains Tundra also includes a Crocodile factory.
- Why: Matches the A3 mapping (“Meadows in Forest” and “Tundra in Plains” now spawn crocs). Done via `.addFactory(() -> new Crocodile(...))`, keeping spawner code closed to modification but open to extension (OCP).

3) Introduced a new spawner type: Swamp (~)

- Change: Swamp extends SpawnerGround with a policy “if any actor is nearby, 50% chance to spawn”. Newborn animals from swamps are immediately poisoned for 10 turns × 5 dmg/turn.
- Why: Fulfills A3’s swamp behaviour. Implemented by composing:

- A proximity-gated spawn policy (SRP: policy decides *when*),
- A newborn-only poison effect (SRP: effect decides *what to do* to the newborn).
- Benefit: No duplication in other spawners; swamps stay a thin configuration wrapper (OCP, SRP).

4) Generalised “extras” into location-aware effects (SpawnContextEffect)

- Change: We split effects into:
 - SpawnEffect (mutate newborn before placement), and
 - SpawnContextEffect (needs the spawner Location, runs after successful placement).
- Why: A3 adds four cross-spawner rules that act around the tile (drop apple/yewberries, grow a tree, AOE poison neighbours). Those require knowledge of exits/nearby actors and must only run if placement succeeded.
- Benefit: Clean separation of *when/where* vs *what*, prevents item spam when placement fails, and lets us apply the same extras to any spawner by just registering the effects once (SRP, OCP, DIP).

5) Implemented the four cross-spawner extras as reusable effects

- Change:
 - Deer → Apple on one exit (DeerDropAppleEffect)
 - Bear → YewBerry scatter (50% each exit) (BearScatterYewEffect)
 - Wolf → Proximity YewBerry Tree on one exit (WolfGrowYewTreeEffect)
 - Crocodile → Poison nearby actors for 3 turns × 10 dmg (CrocodilePoisonAuraEffect)
- Why: A3 requires these to apply regardless of spawner type. We attach them once to each spawner instance in Earth, not inside species or terrains. This keeps animals and terrains independent, and makes future additions one-line registrations (OCP, DIP).

6) Minimal, non-breaking UI feedback

- Change: Added concise `Display.println(...)` messages in effects (e.g., “Deer spawned → dropped Apple at (x,y)”, “Croc spawned → poisoned 3 actors”).
- Why: Required for handover/demo clarity; no engine modification. Improves observability without coupling game rules to presentation.

7) No engine modifications; safe failure handling preserved

- Change: `SpawnerGround.tick` still catches placement exceptions and only runs context effects after successful `addActor`.
- Why: Prevents “ghost” side-effects (e.g., berries dropping when the spawn failed). Keeps the engine contract intact (KISS, robustness).

Why these changes are justified

- They map 1-to-1 to new A3 requirements (crocodile, swamp semantics, and four extras).
- They preserve and strengthen A2 architecture. We didn’t shove more logic into animals or terrains; we extended by composition (policies/effects) as intended (OCP, SRP).
- They reduce future cost. New terrains or rules are just new effects/policies wired in Earth. No ripple edits across species or engine (DIP, LSP).
- They are easy to test and demo. Effects are small, pure(ish) units with clear terminal output; spawners can be validated in isolation.

In short, every change was the minimal extension needed to satisfy A3 while keeping the A2 design principles intact—and in several places, made the design *more* modular by splitting newborn-only vs location-aware effects.